MCMP117 · WEEK 11

Fundamentals of Operating Systems

Week 11 · Permissions, Ownership & Users

Who may read, change or run your files — and how does Linux decide?

Electronical Communication Technology Programme · Vocational School · Istanbul Okan University

LAST WEEK

Week 10 in Four Ideas

Create, copy, move

mkdir -p, touch, cp -r, mv, rm -r, rmdir. rm has no recycle bin.

Wildcards

* ? [abc] and {a,b}: expanded by the shell before the command runs.

Look inside

cat, less, head, tail -f, wc, file, du -sh, df -h.

Inodes & links

stat shows the inode; ln adds a name, ln -s stores a path.

stat printed Access: (0664/-rw-rw-r--) — today we learn to read both halves of that line and to change them.

Today's Agenda

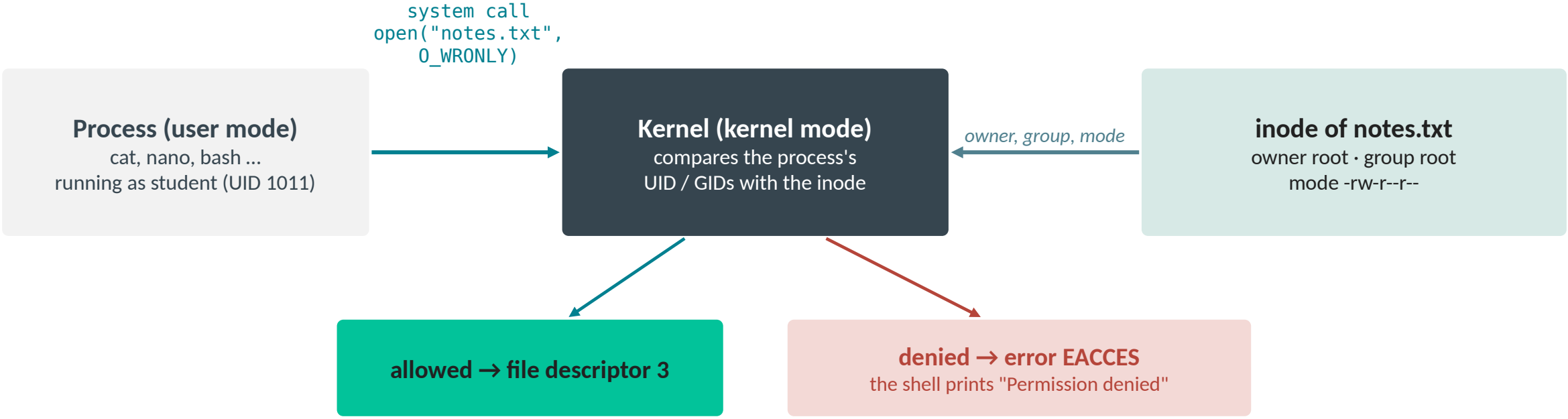
1. **Why, and for whom** Protection (Weeks 2 and 7), users, groups, /etc/passwd and /etc/group.
2. **Reading permissions** The 10-character mode string, r w x for files and directories.
3. **Changing permissions** chmod in symbolic (u+x) and octal (754) form; making a script executable.
4. **Defaults and directories** umask, and what r, w, x really mean for a directory.
5. **Ownership and root** chown, chgrp, sharing with a group, sudo, setuid and the sticky bit.
6. **Lab** Secure a project folder; 16 graded exercises in the notes.

PART 1

Users, Groups and Protection

Every file has an owner; every process runs as somebody.

The Kernel Checks Every open() — It Cannot Be Skipped



Week 2: dual mode

Only the kernel may touch the disk. A program must ask with a system call, so the check cannot be bypassed.

Week 7: file attributes

"Protection" is an attribute in the inode, next to owner, group, size and timestamps.

Today: the Linux rule

Three classes (owner, group, others) × three rights (read, write, execute) = 9 bits.

Who Am I? Users, Groups and Their Numbers

User

A login name plus a number, the UID. The kernel only stores and compares the number. root (UID 0) is the superuser; system users run services.

Group

A named set of users, with a number (GID). Each user has one primary group and may have supplementary groups. Ubuntu gives every user a private group with the same name.

Credentials of a process

Every process carries the UID and GIDs of whoever started it. A new file gets your UID and your primary GID.

```
student@okan-lab:~/lab$ whoami
student
student@okan-lab:~/lab$ id
uid=1011(student) gid=1013(student) groups=1013(student)
student@okan-lab:~/lab$ groups
student
```

Reading id

- **uid=1011(student):** my number and name.
- **gid=1013(student):** my primary group.
- **groups=…:** every group I belong to.

/etc/passwd and /etc/group: One Line per Account

/etc/passwd — 7 fields separated by colons

student	:	x	:	1011	:	1013	:	(empty)	:	/home/student	:	/bin/bash
1 user name		2 password: x = in /etc/shadow		3 UID		4 GID (primary group)		5 comment (full name)		6 home directory		7 login shell

/etc/group — 4 fields

students	:	x	:	1010	:	(empty)
1 group name		2 password (unused)		3 GID		4 members, comma-separated

```
student@okan-lab:~/lab$ ls -l /etc/passwd /etc/group /etc/shadow
-rw-r--r-- 1 root root    809 Oct 12 11:50 /etc/group
-rw-r--r-- 1 root root   1598 Oct 12 11:50 /etc/passwd
-rw-r----- 1 root shadow  884 Oct 12 11:50 /etc/shadow
student@okan-lab:~/lab$ cat /etc/shadow
cat: /etc/shadow: Permission denied
```

Why two files?

- Everybody must read /etc/passwd (ls needs it to turn UIDs into names).
- Password hashes are kept in /etc/shadow: readable only by root and group shadow.

READING LS -L

Seven Columns of ls -l

One line of ls -l docs from our lab folder:

-rw-rw-r--	1	student	student	578	Oct 12 19:38	syllabus.txt
1	2	3	4	5	6	7
type + permissions	links	owner	group	size (bytes)	last modified	name

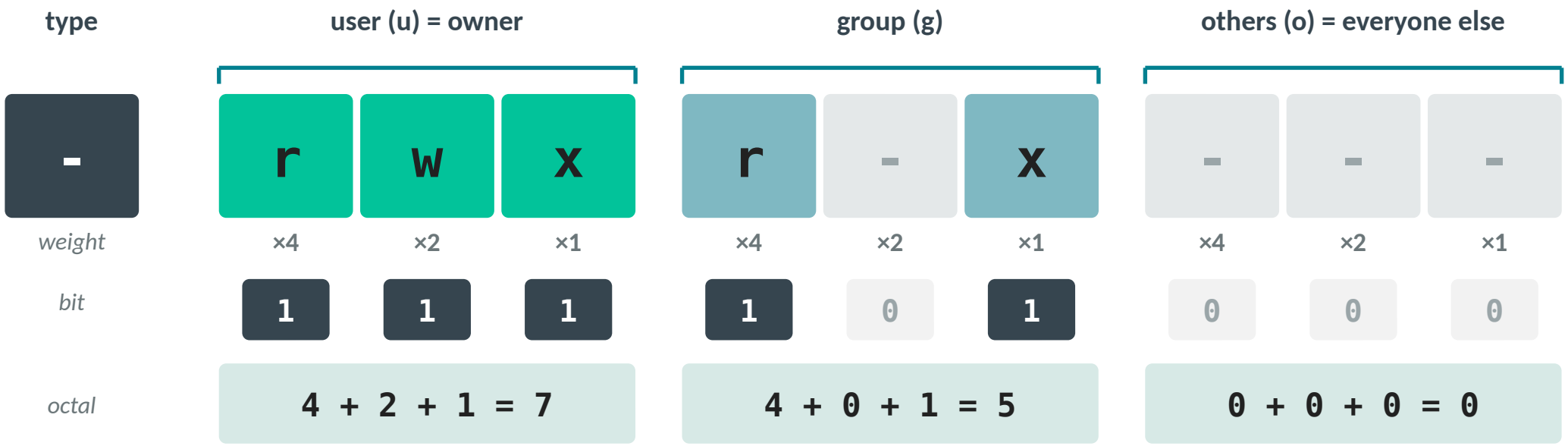
Columns 1–2

- **1 Mode:** file type + 9 permission bits — the next slide.
- **2 Link count:** number of hard links (Week 7, Week 10).

Columns 3–7

- **3, 4 Owner and group:** the inode stores numbers; ls shows the names.
- **5–6:** size in bytes and modification time (mtime).
- **7 Name:** comes from the directory entry, not the inode.

Decoding -rwxr-x--- Character by Character



-rwxr-x--- = 750: owner may read, write and execute · group may read and execute · others: nothing

- **Type letter:** - regular file d directory l symbolic link c / b device

- **Counting:** 3 bits per class → $2^3 = 8$ combinations → exactly one octal digit (0–7).

The Same Three Letters Mean Different Things

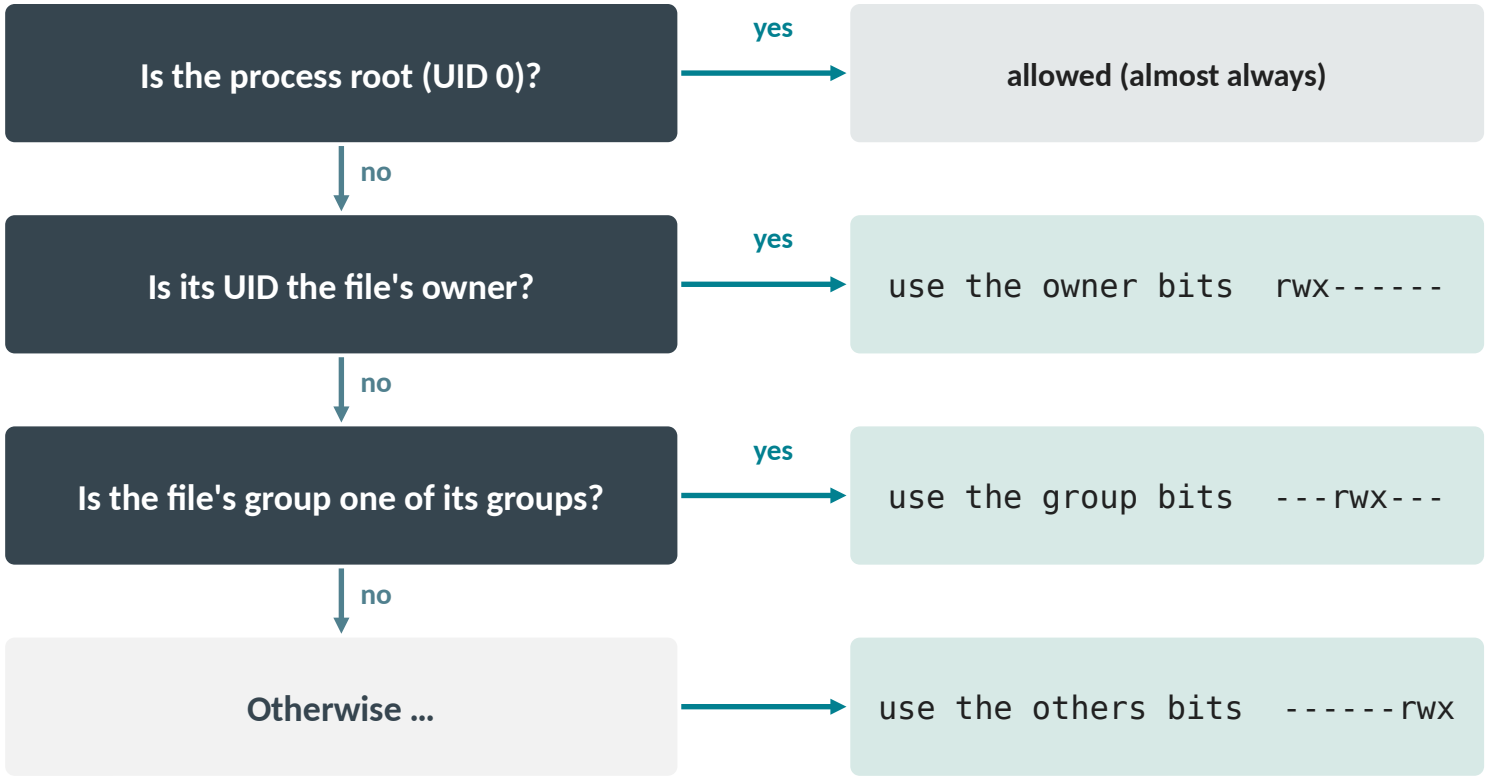
Bit	On a regular file	On a directory
r (4)	read the contents: cat, less, cp	list the names inside: ls
w (2)	change the contents: nano, echo >>	create, delete, rename entries in it (needs x too)
x (1)	run it as a program or script: ./hello.sh	enter it and reach files inside: cd, cat dir/file

The directory x bit is the key to the door

Without x on a directory you cannot cd into it or open anything below it, even if you know the exact name and the file itself is readable. Without r you cannot list it, but with x you can still open a file whose name you know.

Deleting a file is a change to its DIRECTORY, not to the file: you need w + x on the directory — the file's own w bit does not matter.

Only One Class Applies: Owner, Else Group, Else Others



The search stops at the first match.

```
student@okan-lab:~/lab$ echo "hello" > me.txt
student@okan-lab:~/lab$ chmod 074 me.txt
student@okan-lab:~/lab$ ls -l me.txt
----rwxr-- 1 student student 6 Oct 12 19:38 me.txt
student@okan-lab:~/lab$ cat me.txt
cat: me.txt: Permission denied
student@okan-lab:~/lab$ chmod 644 me.txt
student@okan-lab:~/lab$ cat me.txt
hello
```

Surprise

The owner bits are ---, so the owner is refused even though everybody else may read. The kernel never "falls through" to the other classes.

LIVE DEMO

Reading Permissions with ls -l, ls -ld and stat

```
student@okan-lab:~/lab$ ls -l docs
total 24
-rw-rw-r-- 1 student student  53 Oct 12 19:38 'final report.txt'
-rw-rw-r-- 1 student student 302 Oct 12 19:38  linux_quotes.txt
-rw-rw-r-- 1 student student  16 Oct 12 19:38  notes_01.txt
-rw-rw-r-- 1 student student  16 Oct 12 19:38  notes_02.txt
-rw-rw-r-- 1 student student  16 Oct 12 19:38  notes_03.txt
-rw-rw-r-- 1 student student 578 Oct 12 19:38  syllabus.txt
student@okan-lab:~/lab$ ls -ld docs
drwxrwxr-x 2 student student 4096 Oct 12 19:38 docs
student@okan-lab:~/lab$ stat docs/syllabus.txt
  File: docs/syllabus.txt
  Size: 578           Blocks: 8          IO Block: 4096   regular file
Device: 254,0         Inode: 804012      Links: 1
Access: (0664/-rw-rw-r--)  Uid: ( 1011/ student)   Gid: ( 1013/ student)
...
student@okan-lab:~/lab$ stat -c '%A  %a  %U  %G  %n' docs/syllabus.txt docs src/hello.c
-rw-rw-r--  664  student  student  docs/syllabus.txt
drwxrwxr-x  775  student  student  docs
-rw-rw-r--  664  student  student  src/hello.c
```

What to observe

- ls -l docs lists the files INSIDE; ls -ld docs shows the directory itself.
- stat prints the mode twice: 0664 and -rw-rw-r--.
- stat -c with %A %a %U %G %n prints just what we need.
- New files: 664, new directories: 775 — why? See umask, Part 4.

PART 2

Changing Permissions: chmod

Two ways to say the same thing: letters (u+x) or numbers (754).

chmod who operator permissions file …

who

- **u** — user = owner
- **g** — group
- **o** — others
- **a** — all (= ugo)

operator

- **+** — add
- **-** — remove
- **=** — set exactly

permissions

- **r** — read
- **w** — write
- **x** — execute

Command (on syllabus.txt)	Before	After
`chmod u+x`	`rw-rw-r--`	**`rwxrw-r--`**
`chmod go-w`	`rwxrw-r--`	**`rwxr--r--`**
`chmod a=r`	`rwxr--r--`	**`r--r--r--`**
`chmod u=rw,g=r,o=`	`r--r--r--`	**`rw-r-----`**
`chmod u+rwx,g+w`	`rw-r-----`	**`rwxrw----`**

Combine with commas, no spaces: `u=rw,g=r,o=`

+ and − change only the bits you name; = replaces the whole triplet (o= with nothing after it means "no rights for others"). Symbolic mode is best for small changes: "also let the group write" → chmod g+w.

LIVE DEMO

chmod in Symbolic Mode

```
student@okan-lab:~/lab$ cd docs
student@okan-lab:~/lab/docs$ ls -l syllabus.txt
-rw-rw-r-- 1 student student 578 Oct 12 19:38 syllabus.txt
student@okan-lab:~/lab/docs$ chmod u+x syllabus.txt
student@okan-lab:~/lab/docs$ ls -l syllabus.txt
-rwxrw-r-- 1 student student 578 Oct 12 19:38 syllabus.txt
student@okan-lab:~/lab/docs$ chmod go-w syllabus.txt
student@okan-lab:~/lab/docs$ ls -l syllabus.txt
-rwxr--r-- 1 student student 578 Oct 12 19:38 syllabus.txt
student@okan-lab:~/lab/docs$ chmod a=r syllabus.txt
student@okan-lab:~/lab/docs$ ls -l syllabus.txt
-r--r--r-- 1 student student 578 Oct 12 19:38 syllabus.txt
student@okan-lab:~/lab/docs$ chmod u=rw,g=r,o= syllabus.txt
student@okan-lab:~/lab/docs$ ls -l syllabus.txt
-rw-r----- 1 student student 578 Oct 12 19:38 syllabus.txt
student@okan-lab:~/lab/docs$ chmod u+rwx,g+w syllabus.txt
student@okan-lab:~/lab/docs$ ls -l syllabus.txt
-rwxrw---- 1 student student 578 Oct 12 19:38 syllabus.txt
```

What to observe

- chmod prints nothing when it succeeds — check with ls -l.
- u+x and go-w leave all other bits alone.
- a=r overwrote everything: even the owner lost w.
- The owner can still chmod a read-only file: changing the mode needs ownership, not w.

Octal: One Digit per Class, r = 4, w = 2, x = 1

Octal	Binary r w x	Symbolic	Typical meaning
0	0 0 0	---	no access
1	0 0 1	--x	execute only
2	0 1 0	-w-	write only (rare)
3	0 1 1	-wx	write + execute (rare)
4	1 0 0	r--	read only
5	1 0 1	r-x	read + execute
6	1 1 0	rw-	read + write
7	1 1 1	rwx	everything

Why octal?

- Each class has 3 bits → $2^3 = 8$ values → exactly one octal digit.
- Nine bits = three digits: user, group, others.

```
# symbolic → octal
rw-   r--   ---
110   100   000
 6     4     0     → 640

# octal → symbolic: 5 = 4 + 1
5 → r-x
```

Converting in Both Directions

Symbolic → octal: add the weights in each triplet

```
rwxr-xr--
rwx = 4 + 2 + 1 = 7
r-x = 4 + 0 + 1 = 5
r-- = 4 + 0 + 0 = 4        → 754

rw-rw-r--  = 6 6 4          → 664
```

Octal → symbolic: split each digit into 4, 2, 1

```
711
7 = 4 + 2 + 1 → rwx
1 =         1 → --x
1 =         1 → --x        → rwx--x--x

640 → rw-  r--  ---        → rw-r-----
```

Quick check: an odd digit always contains x, a digit ≥ 4 always contains r. So 7 5 0 → rwx r-x --- without any arithmetic.

Modes You Will Meet Again and Again

Octal	Symbolic	Typical use
644	rw-r--r--	ordinary files: everyone may read, only the owner writes
755	rwxr-xr-x	programs, scripts and directories that others may use
600	rw-------	private files: passwords, SSH keys (ssh refuses keys that others can read)
700	rwx------	private directories (your home, ~/.ssh)
640 / 750	rw-r----- / rwxr-x---	shared with the group, hidden from everyone else
777	rwxrwxrwx	anyone may change or delete — almost never correct

Careful: chmod 777 "fixes" a Permission denied by giving every user on the machine write access: any user or compromised program can now change or replace the file. Find the one missing bit and add only that (e.g. chmod u+x).

LIVE DEMO

chmod in Octal Mode — and Two Typical Errors

```
student@okan-lab:~/lab$ cd docs
student@okan-lab:~/lab/docs$ chmod 754 syllabus.txt
student@okan-lab:~/lab/docs$ ls -l syllabus.txt
-rwxr-xr-- 1 student student 578 Oct 12 19:38 syllabus.txt
student@okan-lab:~/lab/docs$ chmod 640 syllabus.txt
student@okan-lab:~/lab/docs$ chmod 600 notes_01.txt
student@okan-lab:~/lab/docs$ chmod 644 notes_02.txt
student@okan-lab:~/lab/docs$ chmod 700 ../backup
student@okan-lab:~/lab/docs$ stat -c '%a  %A  %n' syllabus.txt notes_01.txt notes_02.txt ../backup
640  -rw-r-----  syllabus.txt
600  -rw-------  notes_01.txt
644  -rw-r--r--  notes_02.txt
700  drwx------  ../backup
student@okan-lab:~/lab/docs$ chmod 854 notes_03.txt
chmod: invalid mode: ‘854’
Try 'chmod --help' for more information.
student@okan-lab:~/lab/docs$ chmod rw notes_03.txt
chmod: invalid mode: ‘rw’
Try 'chmod --help' for more information.
```

What to observe

- 754 → rwxr-xr--: digits map to triplets.
- One chmod can take several files.
- 700 on a directory: nobody else may even enter.
- 8 and 9 are not octal digits.
- "rw" lacks who/operator: write u+rw or a number.

LIVE DEMO

Making a Script Executable

```
student@okan-lab:~/lab$ cd scripts
student@okan-lab:~/lab/scripts$ echo '#!/bin/bash' > hello.sh
student@okan-lab:~/lab/scripts$ echo 'echo "Hello, $USER! You are in $PWD"' >> hello.sh
student@okan-lab:~/lab/scripts$ cat hello.sh
#!/bin/bash
echo "Hello, $USER! You are in $PWD"
student@okan-lab:~/lab/scripts$ ls -l hello.sh
-rw-rw-r-- 1 student student 49 Oct 12 19:38 hello.sh
student@okan-lab:~/lab/scripts$ ./hello.sh
bash: ./hello.sh: Permission denied
student@okan-lab:~/lab/scripts$ bash hello.sh
Hello, student! You are in /home/student/lab/scripts
student@okan-lab:~/lab/scripts$ chmod u+x hello.sh
student@okan-lab:~/lab/scripts$ ls -l hello.sh
-rwxrw-r-- 1 student student 49 Oct 12 19:38 hello.sh
student@okan-lab:~/lab/scripts$ ./hello.sh
Hello, student! You are in /home/student/lab/scripts
student@okan-lab:~/lab/scripts$ hello.sh
bash: hello.sh: command not found
student@okan-lab:~/lab/scripts$ cd ../src
student@okan-lab:~/lab/src$ gcc -Wall -O2 hello.c -o hello
student@okan-lab:~/lab/src$ ls -l hello.c hello
-rwxrwxr-x 1 student student 15960 Oct 12 19:38 hello
-rw-rw-r-- 1 student student    90 Oct 12 19:38 hello.c
student@okan-lab:~/lab/src$ ./hello
Hello from the lab!
```

What to observe

- ./hello.sh needs x; bash hello.sh only needs r (bash reads it).
- hello.sh alone is not found: the current directory is not in PATH, so type ./
- gcc sets x itself: 775 = 777 minus the umask 002.

Week 14 preview

#!/bin/bash (the "shebang") tells the kernel which interpreter runs the file.

CHEAT SHEET

Commands So Far

Command	What it does	Example
whoami, id, groups	who am I, my UID/GID, my groups	id
ls -l / ls -ld dir	mode, owner, group of files / of the directory itself	ls -ld ~/lab/docs
stat -c '%a %A %n'	octal and symbolic mode, name	stat -c '%a %A %n' *
chmod who±=perm	symbolic change	chmod u+x,go-w run.sh
chmod NNN	octal: set all nine bits	chmod 640 report.txt
chmod -R	recursive: a whole tree (careful!)	chmod -R go-rwx project
./script.sh	run a file from the current directory	./hello.sh

chmod -R applies the same mode to files AND directories: chmod -R 644 dir removes x from every subdirectory and locks you out. Use symbolic mode (go-rwx) with -R.

INTERMISSION

10-Minute Break

Next: default permissions (umask), directories, ownership, root and sudo.

PART 3

Defaults, Directories, Owners and root

Where 664 comes from, and who may change owners.

umask: the Bits to Take Away from New Files

Programs ask for 666 (files) or 777 (directories). The kernel then clears every bit that is set in the process's umask.



In words

- umask = "mask" of bits never given out by default.
- It is a per-process setting, inherited by children.
- It can only remove bits, never add them.

Rule: new mode = requested AND (NOT umask). Files are requested as 666 — never with x — so no new file is executable until you chmod it. Typical defaults: 002 for Ubuntu users (→ 664 / 775), 022 for root and many servers (→ 644 / 755).

umask Values — and Why Subtraction Can Be Wrong

umask	New file (666)		New directory (777)		Used for
002	664	rw-rw-r--	775	rwxrwxr-x	Ubuntu users (private groups)
022	644	rw-r--r--	755	rwxr-xr-x	root, many servers
027	640	rw-r-----	750	rwxr-x---	group may read, others nothing
077	600	rw-------	700	rwx------	fully private
033	**644**	**rw-r--r--**	744	rwxr--r--	the subtraction trap →

```
# umask 033 on a file
  666 = 110 110 110
  033 = 000 011 011
 ~033 = 111 100 100
  AND = 110 100 100
      = 644  rw-r--r--

# 666 − 033 = 633  ✗
# 633 = rw--wx-wx
```

Subtraction works only when every umask bit is also set in 666 or 777. For files, 666 has no x bits, so a umask that contains x bits (033, 027, 077 …) gives a wrong answer by subtraction: 666 − 033 = 633 would mean "write and execute but not read". The kernel just clears bits: 644.

LIVE DEMO

umask in Action

```
student@okan-lab:~/lab/umask_test$ umask
0002
student@okan-lab:~/lab/umask_test$ umask -S
u=rwx,g=rwx,o=rx
student@okan-lab:~/lab/umask_test$ touch f_002; mkdir d_002   # … also 022, 077, 033
student@okan-lab:~/lab/umask_test$ stat -c '%a  %A  %n' f_* d_*
664  -rw-rw-r--  f_002
644  -rw-r--r--  f_022
644  -rw-r--r--  f_033
600  -rw-------  f_077
775  drwxrwxr-x  d_002
755  drwxr-xr-x  d_022
744  drwxr--r--  d_033
700  drwx------  d_077
```

What to observe

- umask -S shows what is ALLOWED: u=rwx,g=rwx,o=rx.
- Files never get x by default.
- 033 → file 644, not 633; directory 744.
- umask 077 → only you can see new files.

umask changes only the current shell (and programs started from it). Put umask 027 in ~/.bashrc to make it permanent. Existing files are not affected.

LIVE DEMO

Directories Without x or Without r

```
student@okan-lab:~/lab$ mkdir box
student@okan-lab:~/lab$ echo "top secret" > box/secret.txt
student@okan-lab:~/lab$ ls -ld box
drwxrwxr-x 2 student student 4096 Oct 12 19:38 box
student@okan-lab:~/lab$ chmod u-x box
student@okan-lab:~/lab$ ls -ld box
drw-rwxr-x 2 student student 4096 Oct 12 19:38 box
student@okan-lab:~/lab$ ls box
secret.txt
student@okan-lab:~/lab$ ls -l box
ls: cannot access 'box/secret.txt': Permission denied
total 0
-????????? ? ? ? ?            ? secret.txt
student@okan-lab:~/lab$ cd box
bash: cd: box: Permission denied
student@okan-lab:~/lab$ cat box/secret.txt
cat: box/secret.txt: Permission denied
student@okan-lab:~/lab$ chmod u-r,u+x box
student@okan-lab:~/lab$ ls -ld box
d-wxrwxr-x 2 student student 4096 Oct 12 19:38 box
student@okan-lab:~/lab$ ls box
ls: cannot open directory 'box': Permission denied
student@okan-lab:~/lab$ cat box/secret.txt
top secret
```

What to observe

- **r but no x:** ls shows the names, but ls -l cannot read the inodes (??????), cd and cat fail.
- **x but no r:** cannot list the directory, but a file whose name you know can be opened.
- Usual pairs on directories: r-x or rwx — r and x go together.

LIVE DEMO

Deleting a Read-Only File: w on the Directory Decides

```
student@okan-lab:~/lab$ chmod 755 box
student@okan-lab:~/lab$ chmod 444 box/secret.txt
student@okan-lab:~/lab$ ls -l box
total 4
-r--r--r-- 1 student student 11 Oct 12 19:38 secret.txt
student@okan-lab:~/lab$ echo "more" >> box/secret.txt
bash: box/secret.txt: Permission denied
student@okan-lab:~/lab$ rm box/secret.txt
rm: remove write-protected regular file 'box/secret.txt'? y
student@okan-lab:~/lab$ ls box
student@okan-lab:~/lab$ chmod 555 box
student@okan-lab:~/lab$ touch box/new.txt
touch: cannot touch 'box/new.txt': Permission denied
student@okan-lab:~/lab$ chmod 755 box
```

What to observe

- 444 stops writing into the file …
- … but rm only asks a question and then deletes it: the directory (755) is writable.
- Directory 555: nothing can be added or removed.

Careful: rm -f skips that question, and rm -r has no recycle bin. A read-only file is NOT protected against deletion.

chown and chgrp: Who May Change the Owner?

The commands

- **chown user file** — change the owner (also chown user:group file).
- **chgrp group file** — change the group.
- Both accept -R for a whole tree.

The rules on Linux

- **chown:** only root. Otherwise you could give files away to dodge disk quotas, or hand someone a file they never created.
- **chgrp:** the owner may switch to a group he or she belongs to.

```
student@okan-lab:~/lab$ ls -l docs/notes_01.txt
-rw-rw-r-- 1 student student 16 Oct 12 19:38 docs/notes_01.txt
student@okan-lab:~/lab$ chown root docs/notes_01.txt
chown: changing ownership of 'docs/notes_01.txt': Operation not permitted
student@okan-lab:~/lab$ chgrp students docs/notes_01.txt
chgrp: changing group of 'docs/notes_01.txt': Operation not permitted
student@okan-lab:~/lab$ chown $USER docs/notes_01.txt
```

Reading the errors

- Operation not permitted (EPERM): you lack the privilege — not a missing r/w/x bit.
- chgrp failed: student is not (yet) in the group students.

Sharing a Folder with a Group

```
student@okan-lab:~/lab$ id
uid=1011(student) gid=1013(student) groups=1013(student),1010(students)
student@okan-lab:~/lab$ groups
student students
student@okan-lab:~/lab$ chgrp students docs/notes_01.txt
student@okan-lab:~/lab$ ls -l docs/notes_01.txt
-rw-rw-r-- 1 student students 16 Oct 12 19:38 docs/notes_01.txt
student@okan-lab:~/lab$ mkdir shared
student@okan-lab:~/lab$ chgrp students shared
student@okan-lab:~/lab$ chmod 2770 shared
student@okan-lab:~/lab$ ls -ld shared
drwxrws--- 2 student students 4096 Oct 12 19:38 shared
student@okan-lab:~/lab$ touch shared/plan.txt
student@okan-lab:~/lab$ ls -l shared
total 0
-rw-rw-r-- 1 student students 0 Oct 12 19:38 plan.txt
```

What to observe

- After the administrator ran sudo usermod -aG students student and student logged in again, id lists the new group.
- Now chgrp works.
- 2770: the extra 2 is setgid on a directory — new files inherit the group students (shown as s).

Recipe for a team folder: put the members in one group, chgrp the folder to it, chmod 2770 (owner + group everything, others nothing, setgid so new files stay in the group).

root and sudo: Borrowing Superuser Power

root

- UID 0: the permission checks of read and write do not apply.
- Ubuntu has no root password by default; administrators use sudo.

sudo command

- Runs ONE command as root.
- Asks for YOUR password (not root's).
- Only for users allowed in /etc/sudoers (Ubuntu: group sudo).
- Logged.

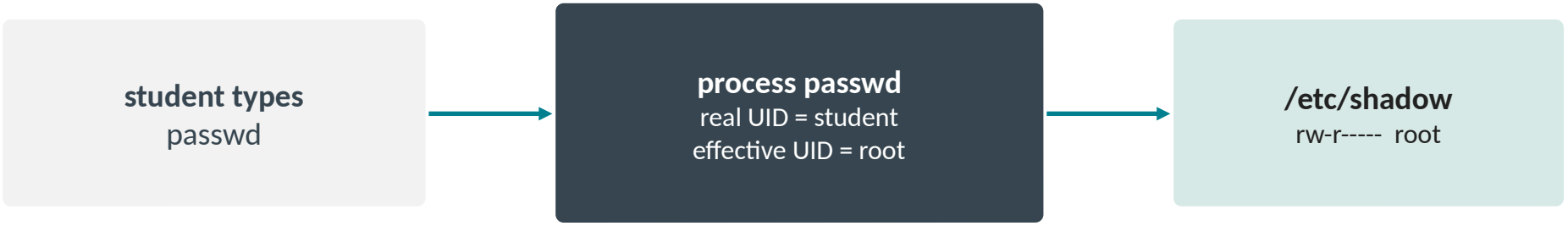
```
student@okan-lab:~/lab$ sudo whoami
[sudo] password for student:
student is not in the sudoers file.
```

student is not an administrator

```
student@okan-lab:~/lab$ groups
student sudo students
student@okan-lab:~/lab$ sudo whoami
[sudo] password for student:
root
student@okan-lab:~/lab$ chown root docs/notes_02.txt
chown: changing ownership of 'docs/notes_02.txt':
Operation not permitted
student@okan-lab:~/lab$ sudo chown root:root
docs/notes_02.txt
student@okan-lab:~/lab$ ls -l docs/notes_02.txt
-rw-rw-r-- 1 student student 16 Oct 12 19:38
docs/notes_02.txt
student@okan-lab:~/lab$ echo "more" >> docs/notes_02.txt
bash: docs/notes_02.txt: Permission denied
```

Careful: Read a sudo command twice before pressing Enter: as root, one typo such as sudo rm -r / home/x (note the space) deletes the whole system. Never run sudo on commands copied from the internet that you do not understand.

setuid: How passwd Can Write /etc/shadow



the s in rwsr-xr-x: run with the FILE OWNER's rights (root)

```
student@okan-lab:~/lab$ ls -l /usr/bin/passwd /etc/shadow
-rw-r----- 1 root shadow   884 Oct 12 11:50 /etc/shadow
-rwsr-xr-x 1 root root   64152 May 30  2024 /usr/bin/passwd
student@okan-lab:~/lab$ stat -c '%a  %A  %n' /usr/bin/passwd /tmp
4755  -rwsr-xr-x  /usr/bin/passwd
1777  drwxrwxrwt  /tmp
```

Octal

- setuid = 4000
- setgid = 2000
- sticky = 1000
- 4755 → rwsr-xr-x

passwd is trusted code that changes only YOUR line. A setuid-root program with a bug gives the attacker root, so there are very few of them (passwd, su, sudo …) and you should never make your own scripts setuid.

The Sticky Bit: Why You Cannot Delete Others' Files in /tmp

```
student@okan-lab:~/lab$ ls -ld /tmp
drwxrwxrwt 16 root root 4096 Oct 12 19:38 /tmp
student@okan-lab:~/lab$ ls -l /tmp/w11_admin.txt
-rw-rw-rw- 1 root root 12 Oct 12 19:38 /tmp/w11_admin.txt
student@okan-lab:~/lab$ rm /tmp/w11_admin.txt
rm: cannot remove '/tmp/w11_admin.txt': Operation not permitted
student@okan-lab:~/lab$ echo "my notes" > /tmp/w11_mine.txt
student@okan-lab:~/lab$ ls -l /tmp/w11_mine.txt
-rw-rw-r-- 1 student student 9 Oct 12 19:38 /tmp/w11_mine.txt
student@okan-lab:~/lab$ rm /tmp/w11_mine.txt
```

What to observe

- /tmp is rwxrwxrwt = 1777: everybody may create files.
- The t (sticky bit): only the file's owner (or root) may delete or rename it.
- Even a file that is writable for all (rw-rw-rw-) cannot be deleted by others.

Without the sticky bit, w on /tmp would let every user delete everyone else's files (slide 28). The t replaces x in the others triplet; a capital T would mean sticky without x.

PART 4

Lab: Secure a Project Folder

30–40 minutes. Full lab sheet with 16 graded exercises in the course notes.

Exercises 1–9: Read, Decode, chmod

A · Who am I? (1–4)

- whoami, id, groups: note UID and groups.
- Your line in /etc/passwd: name the 7 fields.
- Decode ls -l docs/syllabus.txt into octal.
- Convert on paper: rwxr-x---, rw-r--r--, 640, 711, 775.

B · chmod (5–7)

- Create scripts/hello.sh, make it executable for the owner only, run it.
- notes_01.txt → rw-r----- using symbolic mode only.
- notes_02.txt → owner read/write only, using octal.

B · Secure projects (8–9)

- projects/alpha: only you may enter, list or change it.
- projects/beta: group may read and enter, others nothing; README.md → 640.
- Check everything with stat -c '%a %A %n'.

Points: Part A 20, Part B 48, Part C 32 — total 100. Write the command you used AND the ls -l / stat line that proves it worked.

Exercises 10–16: umask, Directories, Owners

B · Defaults (10–12)

- Predict, then verify, the modes of a new file and directory with umask 027.
- Same with umask 033: why not 633?
- Remove x from your own directory: which commands fail?

C · Challenge (13–15)

- Delete a read-only file in your directory. Then make the directory 555 and try again.
- Try chown and chgrp: explain both errors.
- passwd and /tmp: find the s and the t, explain them.

C · Team folder (16)

- projects/team: dir 750, deploy.sh 750, notes.txt 640, secret.txt 600.
- Run ./projects/team/deploy.sh.
- Print all four modes with one stat command.

Finished early? Set umask 077 in a new terminal, create a file, and explain the result to your neighbour using the binary of 666 and 077.

Error Messages and What They Mean

You see	Because	Fix
bash: ./hello.sh: Permission denied	the file has no x bit	chmod u+x hello.sh
bash: hello.sh: command not found	. is not in PATH	type ./hello.sh
bash: cd: box: Permission denied	directory without x	chmod u+x box
ls: cannot open directory 'box'	directory without r	chmod u+r box
chown: ... Operation not permitted	only root may change owners	ask the admin / sudo chown
chmod: invalid mode: '854'	8 and 9 are not octal digits	digits 0–7 only
student is not in the sudoers file.	you are not an administrator	ask the administrator

QUICK QUIZ

Check Your Understanding

1. What does -rwxr-x--- allow, and what is it in octal?

2. Make report.txt read/write for the owner, read-only for the group, nothing for others — in octal and in symbolic form.

3. With umask 027, what are the modes of a new file and a new directory? Why is umask 033 on a file 644 and not 633?

4. A file in your directory has mode r--r--r--. Can you delete it? What about a file you own with mode ----rwx---: can you read it?

WRAP-UP

Week 11 — Key Takeaways

- **Protection:** every open() goes through the kernel (Week 2), which compares the process's UID/GIDs with the owner, group and mode in the inode (Week 7).
- **Mode string:** type + three triplets (user, group, others) of r w x; weights 4 2 1 give one octal digit per class: 750 = rwxr-x---.
- **Only one class applies:** owner, else group, else others.
- **Directories:** r = list, w = create/delete entries, x = enter. Deleting needs w on the directory.
- **chmod:** symbolic (u+x, go-w, a=r) or octal (644, 755, 600). Never 777.
- **umask:** clears bits from 666/777: 002 → 664/775, 022 → 644/755, 033 → 644 (bitwise, not subtraction).
- **Owners and root:** chown needs root; sudo runs one command as root; setuid (passwd) and sticky (/tmp) are special bits.

NEXT WEEK

Text Processing, Redirection & Pipes

Week 12 will cover:

- stdin, stdout, stderr = file descriptors 0, 1, 2 (Weeks 2–3)
- Redirection: `>` `>>` `<` `2>` `2>&1` `/dev/null`
- Pipes: `|` connects processes created by the shell
- grep, sort, uniq, cut, wc, tr, tee, find
- Real analyses of logs/access.log and data/students.csv

QUESTIONS?

Discussion

If you have questions after class, ask them on the course Telegram channel — everyone benefits from the answer.

Asst. Prof. Dr. Hasan Oğuz